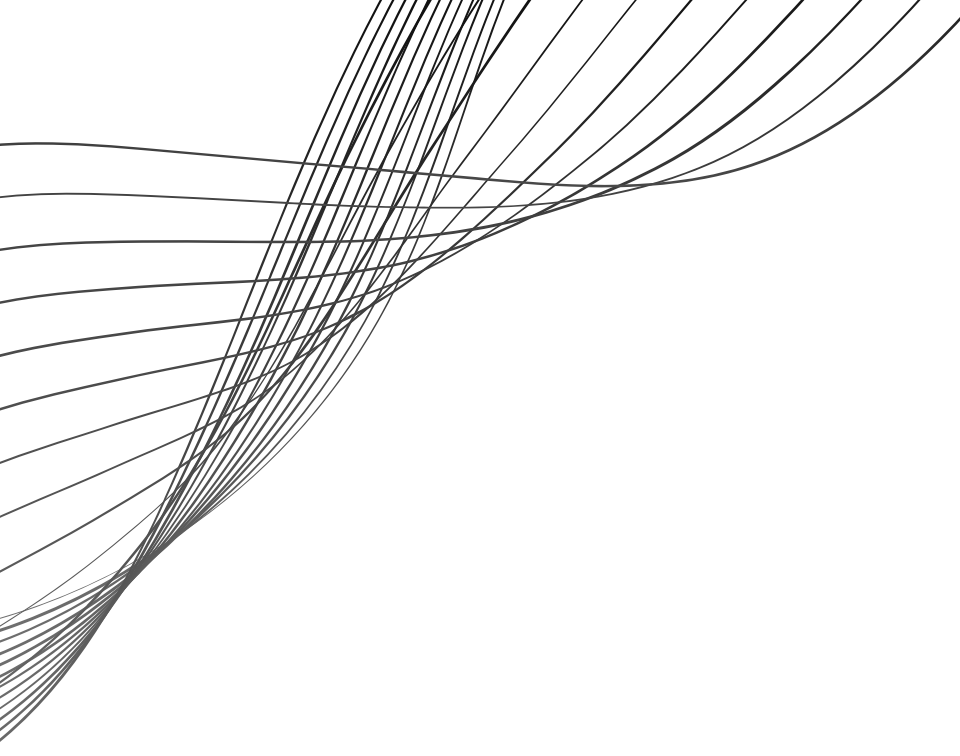


# POO COM C++

Pedro Henrique Ribeiro Dias

Inatel

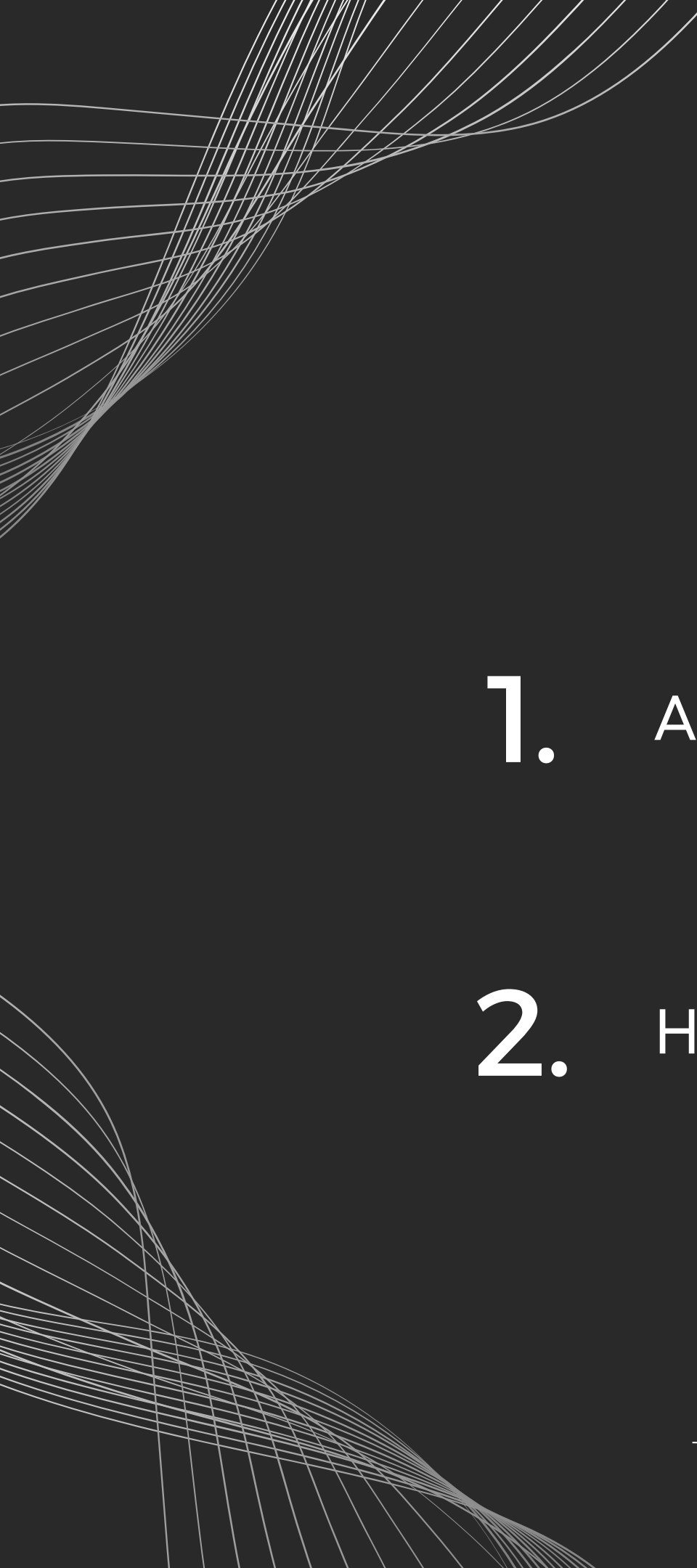


# INTRODUÇÃO

A programação orientada a objetos (POO) organiza código em torno de objetos reais, facilitando a modelagem de sistemas complexos. Podemos especificar as características e comportamentos desses objetos, além de estabelecer relações entre eles.

# POO COM C++

1. Permite melhor organização, reutilização de código e manutenção facilitada, promovendo sistemas altamente escaláveis e flexíveis.
2. Criamos classes como moldes para objetos que possuem atributos e métodos, ou seja, características e comportamentos



# PILARES DA POO

1. Abstração

2. Herança

3. Encapsulamento

4. Polimorfismo

# ABSTRAÇÃO

Abstraímos Entidades do mundo real para nosso código, dando a elas características e comportamentos que representem a realidade.

•Exemplo:

- Identidade : Aluno.
- Propriedades: Nome, Idade, Matrícula.
- Métodos: Questionar, Estudar , Surtar.

# HERANÇA

Uma classe pode "herdar" características de outra. Isso evita repetir código, portanto atributos da classe “mãe” são passados para a classe filha.

## •Exemplo

- Uma classe Professor pode herdar de Ser Humano. Com isso atributos (nome, idade) e métodos (Falar, Comer) da classe Ser Humano seriam passados para Professor.



# ENCAPSULAMENTO

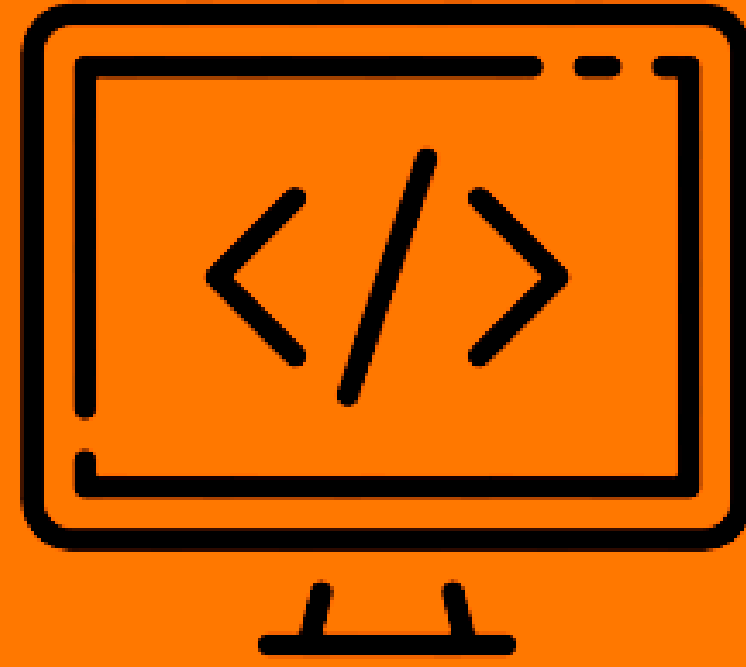
Ao definirmos os atributos de um objeto, devemos garantir que eles sejam "trancados", ou seja, o objeto pode ter suas propriedades consultadas, mas não modificadas por outros objetos.

# POLIMORFISMO

É as características de dois ou mais objetos derivados de uma mesma classe possuírem respostas diferentes para cada método.

Exemplo:

- Classes Animal possui um método para emitir som. Suas classes filhas, como cachorro e gato, teriam esse método diferente, pois emitem diferentes sons.



**KEEP  
CALM**

AND

**PROGRAMME  
ORIENTADO A OBJETOS**

# EXERCÍCIO 1

**Cenário:** Em um grande festival de música com um duelo de bandas, cada banda possui um nome, uma quantidade de integrantes, uma potência de som e um nível de energia da plateia.

**Tarefa:**

- Crie uma classe Banda.
- Adicione os atributos: nome (string), integrantes (int), potenciaSom (float) e energia (int).
- Crie um método chamado duelar(): Ele deve receber como parâmetro um objeto da classe Banda (a rival). O método deve exibir uma mensagem confirmando a apresentação e subtrair o valor da potenciaSom da energia da banda rival.

**Na main:**

- Instancie 2 objetos da classe Banda.
- Atribua valores personalizados aos atributos de cada um.
- Execute o método duelar(), definindo qual banda será a desafiante e qual será a rival.
- Ao final, exiba o status atualizado de ambas as bandas para ver como ficou a energia após o confronto.

# EXERCÍCIO 2

**Cenário:** No jogo Persona, a relação do protagonista com seus aliados (Links Sociais) evolui à medida que passam tempo juntos, alterando seus níveis de afinidade de forma controlada.

**Tarefa:**

- Crie uma classe chamada LinkSocial.
- Adicione os atributos nome (string) (obs: nome do personagem), arcana (string) (obs: arcana que representa o Link entre o protagonista e o personagem) e rank (int) como privados.
- Crie os métodos públicos de acesso (getters) e modificação (setters) para cada atributo.
- Crie um método chamado subirRank(): Ele deve apenas incrementar em +1 o valor do atributo rank.

**Na main:**

- Instancie um objeto da classe LinkSocial.
- Utilize os métodos setters para definir o nome, a arcana e o rank inicial (ex: 1) do personagem.
- Execute o método subirRank().
- Exiba na tela os dados do Link Social utilizando os métodos getters para confirmar o aumento do rank.

# EXERCÍCIO 3

**Cenário:** No Inatel, durante eventos acadêmicos e feiras tecnológicas como a FETIN, diferentes pessoas desempenham papéis essenciais na comunidade. Todos compartilham características de um membro da instituição, mas alunos e professores possuem atribuições e dados específicos dentro do campus.

**Tarefa:**

- Crie uma classe base chamada MembroInatel.
- Adicione o atributo nome (string).
- Crie um método chamado seApresentar() que exiba na tela: "Sou um membro da comunidade Inatel: [nome]."
- Crie as classes Aluno e Professor que herdem de MembroInatel:
  - Adicione o atributo curso (string) na classe Aluno.
  - Adicione o atributo disciplina (string) na classe Professor.

# EXERCÍCIO 3

- Sobrescreva o método seApresentar() em ambas as classes filhas:
  - No Aluno, ele deve exibir: "Meu nome é [nome] e estudo no curso de [curso]."
  - No Professor, ele deve exibir: "Meu nome é [nome] e leciono a disciplina de [disciplina]."

## **Na main:**

- Instancie um objeto da classe Aluno e um objeto da classe Professor.
- Atribua valores aos atributos de cada um deles.
- Chame o método seApresentar() de cada objeto para demonstrar como a classe filha reutilizou e personalizou o comportamento da classe base.

# EXERCÍCIO 4

**Cenário:** Na Comarca, a terra natal dos Hobbits, a vida é tranquila e cheia de tradições cotidianas. Embora todos compartilhem o amor pela paz e pela boa comida, cada Hobbit possui uma ocupação diferente para manter a comunidade funcionando.

**Tarefa:**

- Crie uma classe base chamada Hobbit:
  - Adicione o atributo nome (string).
  - Crie um método (virtual) chamado fazerAtividade(). Dentro dele, imprima: "O hobbit [nome] está aproveitando um dia tranquilo na Comarca."
- Crie três classes filhas que herdem de Hobbit:
  - Jardineiro: Sobrescreva fazerAtividade() para exibir: "O jardineiro [nome] está cuidando das flores e plantas ao redor das tocas!"
  - Cozinheiro: Sobrescreva fazerAtividade() para exibir: "O cozinheiro [nome] está preparando o segundo café da manhã para os convidados!"
  - Fazendeiro: Sobrescreva fazerAtividade() para exibir: "O fazendeiro [nome] está colhendo vegetais e hortaliças em suas terras!"

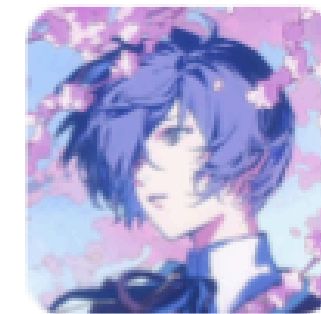
# EXERCÍCIO 4

## Na main:

- Crie um vetor (ou lista) para armazenar os objetos do tipo Hobbit (utilizando ponteiros da classe base).
- Crie uma instância de cada profissão (Jardineiro, Cozinheiro e Fazendeiro) com seus respectivos nomes e adicione-as ao seu vetor.
- Percorra o vetor e, para cada elemento, chame o método fazerAtividade() para demonstrar o comportamento polimórfico.

# FIM DA AULA!

## zSh3rl0cK/S01-monitoria



 1  
Contributor

 0  
Issues

 0  
Stars

 0  
Forks



### zSh3rl0cK/S01-monitoria

Contribute to zSh3rl0cK/S01-monitoria development by creating an account on GitHub.



GitHub